GSoC 2026 · CERN-HSF / ROOT

Keras Conv2DTranspose Parser & NHWC→NCHW Memory Tracking

TMVA::Experimental::SOFIE::PyKeras · Exercise 5 – Operator Expansion

*Author: **Aditya Rathore** · GitHub: @AdityaDRathore*

Submitted: March 2026 · For review by Lorenzo Moneta & Sanjiban Sengupta

§0 Context & Motivation

This document is my technical write-up for **Exercise 5** of the GSoC 2026 CERN-HSF SOFIE evaluation. The exercise asks candidates to extend the SOFIE Keras parser with parsing functionality for new operator types. I chose to implement the **Keras Conv2DTranspose** parser because it sits at a demanding intersection of memory-layout safety, AVX/SIMD vectorisation correctness, and C++ shape-inference responsibility boundaries — all of which are first-class concerns inside SOFIE's strictly-typed, statically-allocated C++ backend.

My prior work on the project — PR #21528, adding 10 new operators to `RModelParser_PyTorch.cxx` — gave me direct exposure to the pybind11/C-API trust boundary and the strict serialisation contracts SOFIE's factory methods impose. That foundation informed every design decision described here, especially the memory-contiguity enforcement and the Global Layout State Tracker. During this work I also identified Issue #21527 (Clean_name() tensor name collisions) and filed PR #21582 (CMake silent exclusion vulnerability); both are documented below.

§1 What Is Conv2DTranspose — and Why Is It Non-Trivial to Parse?

1.1 The Operator

A transposed convolution (fractionally-strided convolution) is the gradient-of-Conv2D operator used to up-sample feature maps in generative networks, segmentation decoders, and physics simulation generators. Given an input $X \in R^{N \times C_{in} \times H_{in} \times W_{in}}$, kernel $W \in R^{C_{in} \times C_{out} \times kH \times kW}$, and stride S, the output spatial dimensions under SAME padding expand as:

$$H_{out} = H_{in} \times S_h,\ W_{out} = W_{in} \times S_w$$

The semantics invert Conv2D's contraction behaviour: each input element `X[n, c_in, h, w]` scatters a scaled copy of the kernel into the output at position `(h·S_h, w·S_w)`. Contributions overlap and are summed, making output shape resolution non-trivial when strides > 1 and padding is not VALID.

1.2 The Memory-Layout Problem

Keras stores all tensors in **NHWC** (channels-last) format by default. SOFIE's C++ backend, following ONNX convention, requires **NCHW** (channels-first). Beyond the input activation, the kernel weight tensor itself has a different axis order between the two frameworks:

Framework	Kernel Layout	Axis Order
Keras (channels_last)	W_Keras ∈ R^(H×W×C_out×C_in)	(H, W, C_out, C_in)
SOFIE / ONNX (channels_first)	W_SOFIE ∈ R^(C_in×C_out×H×W)	(C_in, C_out, H, W)

To bridge these layouts correctly, a precise axes permutation of **(3, 2, 0, 1)** must be applied to the extracted Keras kernel. Getting this wrong — even by swapping C_in and C_out — produces outputs that are numerically plausible but physically incorrect, with no obvious error signal at runtime.

1.3 Why Parsing Is Non-Trivial

Several Keras Conv2DTranspose attributes each change what must appear in the IR:

Keras Attribute	Effect on IR	Parser Action Required
`padding='same'`	Output spatial dims double with stride=2; pads depend on runtime input topology	Emit autopad='SAME_UPPER'; omit static pads
`padding='valid'`	Output grows by (k–1)·S; zero padding on all sides	Set autopad='VALID', pads=[0,0,0,0]
`use_bias=True`	Bias vector of shape (C_out,) present in weights	Extract and ensure C-contiguous float32 bias
`output_padding not None`	Extra rows/cols appended to one side of output	Forward as list; **omit key entirely** when None

§2 Design Decisions — The 'Why'

2.1 The Global Layout State Tracker

Keras aggressively defaults to NHWC. A naïve parser that blindly injects a `Transpose(NHWC→NCHW)` before every Conv2DTranspose and a reverting `Transpose(NCHW→NHWC)` after it would double the memory-copy overhead for every such layer in a multi-layer model. In GANs or UNet decoders — which can have 8–16 Conv2DTranspose layers — that is a ruinous O(N) transpose tax.

The **LayoutTracker** solves this by maintaining a single global `current_layout` string. A Transpose node is injected *only* when the tracker state is `"NHWC"`, and after injection the state is permanently flipped to `"NCHW"` for all downstream layers. A model with 10 Conv2DTranspose layers pays the transposition cost exactly once — at the first layer boundary.

LayoutTracker — global NHWC→NCHW tracking with O(1) transpose cost

```
class LayoutTracker:
    """
    Global Layout State Tracker to prevent redundant Transpose.
    Maintains the current expected dimensional layout of the graph.
    """
    def __init__(self, initial_layout: str = "NHWC"):
        self.current_layout = initial_layout

# In the parser -- emits Transpose only once:
if layout_tracker.current_layout == "NHWC":
    nodes.append({"type": "Transpose", "perm": [0, 3, 1, 2], ...})
    layout_tracker.current_layout = "NCHW"   # permanent state flip
```

2.2 Defer Spatial Mathematics to the C++ ShapeInference Engine

My first draft attempted to statically resolve padding values in Python — computing `pad_h = max((H_in - 1) * S_h + kH - H_out, 0) // 2` and packing the result into the IR dictionary. This was wrong.

The padding required for a `padding='same'` configuration depends on the *runtime input tensor topology* — specifically H_in and W_in, which are not known statically when the parser runs. Hardcoding static padding in Python would corrupt the C++ ShapeInference logic by overriding its dynamic resolution with values computed for a single specific input size.

> **Design principle:** The parser's job is to faithfully transport weights and attributes into the IR. It is not to pre-compute mathematical quantities that the C++ engine is designed to own. When a field's value depends on runtime state, omit it from the Python payload and let the C++ ShapeInference engine resolve it. Violating this boundary produces bugs that only manifest on non-standard input sizes — the worst kind.

2.3 Conditional output_padding Packing

Keras's `output_padding` attribute is `None` by default. When strides > 1, this attribute resolves output shape ambiguities. Forcing it to `[0, 0]` statically (my initial attempt) overwrote the C++ ShapeInference calculation, breaking matrix dimensions for any non-trivial stride configuration.

The fix: the key is **omitted entirely** from the SOFIE payload when `layer.output_padding is None`. The C++ factory treats a missing key as 'resolve dynamically', which is the correct behaviour for all standard Keras Conv2DTranspose configurations.

output_padding — omit rather than default to [0, 0]

```
# Conditional Packing -- defer to C++ ShapeInference when None
if hasattr(layer, 'output_padding') and layer.output_padding is not None:
    if isinstance(layer.output_padding, int):
        parsed['output_padding'] = [layer.output_padding] * 2
    else:
        parsed['output_padding'] = list(layer.output_padding)
# If None: key is simply absent from the dict -- C++ resolves dynamically
```

2.4 Memory Contiguity Enforcement

Calling `np.transpose` returns a *view* with modified metadata strides but leaves the underlying C-buffer fragmented. Passing this directly through the pybind11 bridge to a C++ `float*` causes a catastrophic segmentation fault, because the C++ code assumes a flat memory stride of `sizeof(float)` per element — which strided views violate.

Kernel transposition + contiguity enforcement

```
# Axes permutation: (H, W, C_out, C_in) -> (C_in, C_out, H, W)
transposed_kernel = np.transpose(keras_kernel, (3, 2, 0, 1))

# NOT OPTIONAL -- np.transpose produces a strided view, not a new
# contiguous buffer. AVX/SIMD vectorisation requires contiguous float32
# layout at the C-buffer boundary.
contiguous_kernel = np.ascontiguousarray(transposed_kernel, dtype=np.float32)

# np.ascontiguousarray guarantees:
#   1. C-contiguous layout (stride == sizeof(float) per element)
#   2. dtype exactly float32 (matches float on all target architectures)
#   3. No-op if already contiguous (zero additional cost)
```

2.5 The List[Dict] Return Contract

The parser returns a `List[Dict[str, Any]]` rather than a single `Dict`. This is a deliberate API extension to allow injecting both a **Transpose** AST node and the **ConvTranspose** AST node in a single parser evaluation step.

At the C-API boundary, the ROOT C++ consumer must use `PyList_Check(returned_obj)` and extract nodes via `PyList_GetItem`. Because `PyList_GetItem` returns a **borrowed reference**, explicitly omitting `Py_DECREF` is architecturally correct — calling it would prematurely free memory that the Python runtime still owns, corrupting the ROOT session heap.

§3 Implementation Walkthrough

3.1 Function Signature and Topological Edge Extraction

The function signature mirrors the Keras parser calling convention. Tensor edge names are extracted from the Keras layer's input/output tensors and sanitised for C++ namespace safety — dots, slashes, and colons are all replaced with underscores:

parse_conv2dtranspose_layer — signature and name sanitisation

```
def parse_conv2dtranspose_layer(
    layer,                    # Keras Conv2DTranspose layer instance
    layout_tracker: LayoutTracker
) -> List[Dict[str, Any]]:
    layer_name_clean = layer.name.replace('.', '_')
    input_name  = layer.input.name\
        .replace('.','_').replace('/','_').replace(':','_')
    output_name = layer.output.name\
        .replace('.','_').replace('/','_').replace(':','_')
```

The sanitisation of `':'` characters is particularly important. Keras names tensors with the format `layer_name/output:0`. The unsanitised colon reaches the C++ side where it is interpreted as a namespace separator, causing silent tensor name collisions between layers.

3.2 Weight Extraction and Transposition

Keras's `layer.get_weights()` returns a list where index 0 is always the kernel and index 1 (if present) is the bias. The kernel arrives in Keras-native layout `(H, W, C_out, C_in)`. The required SOFIE layout is `(C_in, C_out, H, W)`. The axes permutation **(3, 2, 0, 1)** achieves this mapping exactly:

Kernel weight extraction and axes permutation

```
# Keras kernel: (H=3, W=3, C_out=2, C_in=1) -- example from test model
keras_kernel = layer.get_weights()[0]
assert keras_kernel.shape == (3, 3, 2, 1)   # (H, W, C_out, C_in)

# Axes mapping: 3->0 (C_in), 2->1 (C_out), 0->2 (H), 1->3 (W)
transposed_kernel = np.transpose(keras_kernel, (3, 2, 0, 1))
assert transposed_kernel.shape == (1, 2, 3, 3)  # (C_in, C_out, H, W)

contiguous_kernel = np.ascontiguousarray(transposed_kernel, dtype=np.float32)
assert contiguous_kernel.flags['C_CONTIGUOUS']  # AVX safety guaranteed
```

3.3 Attribute Extraction and Payload Assembly

All operator attributes are extracted directly from the Keras layer object. Keras layers expose their configuration through well-typed Python properties, making this simpler than the PyTorch ONNX path which requires kind-dispatch on node attributes:

SOFIE IR payload assembly

```
parsed = {
    'type'        : 'ConvTranspose',
    'name'        : layer_name_clean,
    'kernel_shape': list(layer.kernel_size),   # e.g. [3, 3]
    'strides'     : list(layer.strides),       # e.g. [2, 2]
    'dilations'   : list(layer.dilation_rate), # e.g. [1, 1]
    'group'       : 1,                         # standard conv (no depthwise)
    'weight'      : contiguous_kernel,         # C-contiguous (C_in,C_out,H,W)
    'inputs'      : [conv_input_name],
    'outputs'     : [output_name]
}

# Padding attribute dispatch
if layer.padding == 'valid':
    parsed['autopad'] = 'VALID'
    parsed['pads']    = [0, 0, 0, 0]
elif layer.padding == 'same':
    parsed['autopad'] = 'SAME_UPPER'   # defer spatial math to C++ backend

# Bias -- only if the layer uses one
if layer.use_bias and len(layer.get_weights()) > 1:
    parsed['bias'] = np.ascontiguousarray(layer.get_weights()[1], dtype=np.float32)
```

§4 Bugs Encountered During Development

1. C++ Shape Collisions on Strided Convolutions

Root cause: The initial parser design attempted to statically resolve `output_padding=[0, 0]` when Keras returned None. When strides > 1, Keras relies on dynamic output_padding to resolve output shape ambiguities. Forcing it to `[0, 0]` overwrote the C++ ShapeInference logic, breaking matrix dimensions.

> **Fix:** Engineered conditional attribute packing. If output_padding is None in Keras, the key is omitted entirely from the SOFIE payload dictionary. This defers the calculation directly to the C++ backend.

2. The Memory Permutation Trap in Equivalence Testing

Root cause: The numeric equivalence test in C++ failed uniformly across all indices with errors of order 1.0 — not floating-point noise, but fundamental layout mismatches. The test was comparing the C++ engine output (intentionally left in NCHW format by the LayoutTracker) directly against the native Keras evaluation output (NHWC format) inside a flattened 1D loop. Element [0] of the NCHW flattened array corresponds to a completely different spatial position than element [0] of the NHWC flattened array — the strides were completely disjoint.

> **Fix:** Aligned the Keras ground truth to NCHW before numerical comparison by transposing it inside the PyRun_String evaluation: output = numpy.transpose(model(input_data).numpy(), (0, 3, 1, 2)).astype(numpy.float32). Both arrays now share the same dimensional topology before any element-wise comparison is performed.

3. Fake Contract Testing — Black-Box Volume Checks

Root cause: The test suite verified AST injection by checking `EXPECT_EQ(outputConv2DTranspose.size(), pOutputSize)`. This is a black-box volume check: it proves the output array has the right number of elements, but proves nothing about which operators were actually instantiated in the engine's execution DAG. A model that skipped the Transpose node entirely and ran incorrect arithmetic could still produce an output of the correct size.

> **Fix:** Rewrote the C++ test to query the engine's internal RModel directly and assert the exact operator stack: auto operators = r.GetModel().GetOperators(); EXPECT_EQ(operators[0]->Type(), "Transpose"); EXPECT_EQ(operators[1]->Type(), "ConvTranspose"); This test FAILS if the LayoutTracker omitted the Transpose node or injected nodes in the wrong order. Black-box checks would have passed silently.

4. UTILITY::Clean_name() Erases Dots, Causing Tensor Name Collisions — Issue #21527

Root cause: While extending the PyTorch parser (PR #21528), I traced a class of intermittent C++ inference failures back to a bug in `UTILITY::Clean_name()` in `SOFIE_common.cxx` (line 513). The function replaces `'-'` with `'_'` explicitly, but silently *erases* all other non-alphanumeric characters — including dots. TorchScript generates intermediate tensor names using a sequential dotted pattern: `input.1` → `input.2` → `...`. After Clean_name() processes them, `input.1` and `input1` both collapse to `input1` — causing duplicate C++ variable declarations or silent tensor data overwrites. The same collision affects the Keras parser when tensor names contain dots.

UTILITY::Clean_name() — dot-erasure bug and the one-line upstream fix

```
// BEFORE (broken) -- Clean_name() in SOFIE_common.cxx line 513:
std::string UTILITY::Clean_name(std::string input_tensor_name){
    std::string s(input_tensor_name);
    std::replace(s.begin(), s.end(), '-', '_');  // only '-' is replaced
    s.erase(std::remove_if(s.begin(), s.end(),   // '.' is ERASED
        [](char const& c) -> bool {
            return !std::isalnum(c) && c != '_';
        }), s.end());
    return s;  // input.1 -> input1 == input1: COLLISION
}

// AFTER (fix applied in PR #21528 parser-level workaround + proposed
//        upstream fix for SOFIE_common.cxx):
std::replace(s.begin(), s.end(), '-', '_');
std::replace(s.begin(), s.end(), '.', '_');  // <-- add this line
// input.1 -> input_1  (distinct from input1: no collision)
```

Why existing tests passed by coincidence: The existing SEQUENTIAL_MODEL, MODULE_MODEL, and CONVOLUTION_MODEL tests use architectures where TorchScript happens to generate non-colliding name sequences (e.g., input, input.3, input.5 — skipping numbers). Models with Tanh, Softmax, or BatchNormalization trigger the collision because TorchScript assigns consecutive dotted names without gaps. The parser-level workaround in PR #21528 normalises dots to underscores on the Python side before names reach Clean_name(), protecting all operators I added. Issue #21527 tracks the root fix in Clean_name() itself.

5. CMake Silent Exclusion Vulnerability — PR #21582

Root cause: While validating the C++ GTest suites for this Keras parser, I uncovered a critical infrastructure flaw in `tmva/sofie/test/CMakeLists.txt`. The TMVA test configurations gated GTest targets strictly on `if(BLAS_FOUND)`. However, when ROOT falls back to the builtin GSL library (`-Dbuiltin_gsl=ON`), the `use_gsl_cblas` flag is correctly set but `BLAS_FOUND` remains OFF. Consequently, the entire SOFIE execution test suite — including TestRModelParserKeras — was being silently excluded from the build manifest. The CI pipeline was generating **false-positive green builds** while skipping all execution tests on nodes without external BLAS.

CMakeLists.txt — PR #21582 upstream patch

```
# BEFORE (broken -- silent exclusion on builtin_gsl nodes):
if (tpython AND ROOT_KERAS_FOUND AND BLAS_FOUND)

# AFTER (PR #21582 -- accounts for both BLAS and GSL CBLAS fallback):
if (tpython AND ROOT_KERAS_FOUND AND (BLAS_FOUND OR use_gsl_cblas))

# Effect: GTest targets now correctly included in build manifest on both
# external-BLAS and builtin-GSL nodes. CI false-positives eliminated.
```

This was a one-line fix in CMakeLists.txt but its impact is systemic: every developer building ROOT with `-Dbuiltin_gsl=ON` was running SOFIE without execution test coverage. The patch restores correctness guarantees for the entire test infrastructure.

§5 Validation & Test Coverage

5.1 Python Unit Tests — parse_conv2dtranspose.py

The parser module includes a self-contained test suite in its `__main__` block that exercises the full parsing pipeline on a real Keras model:

Test Case	What It Validates	Result
Layout state transition	Transpose node injected exactly once; tracker flips to NCHW permanently	**PASS**
Kernel weight permutation	Output shape (C_in, C_out, H, W) = (3, 8, 3, 3) for a (3→8, 3×3) layer	**PASS**
Memory contiguity	weight.flags['C_CONTIGUOUS'] and bias.flags['C_CONTIGUOUS'] both True	**PASS**
Padding attribute dispatch	autopad='SAME_UPPER' for same; pads=[0,0,0,0]+autopad='VALID' for valid	**PASS**
output_padding omission	Key absent from payload when Keras attribute is None	**PASS**

5.2 Numeric Equivalence Verification — test_sofie_numeric_eval.py

A four-phase numeric equivalence test validates the full mathematical pipeline from Keras NHWC evaluation through parser extraction to simulated NCHW convolution transpose, with a rigorous `1e-5` floating-point tolerance:

test_sofie_numeric_eval.py — four-phase pipeline verification

```
Phase 1: Keras Native Execution (NHWC)
  model.predict(ones_nhwc) -> keras_output_nhwc

Phase 2: PyMVA Extraction
  parse_conv2dtranspose_layer() -> [Transpose node, ConvTranspose node]
  Verify perm=[0,3,1,2], extract weights_nchw and strides

Phase 3: Simulated NCHW Execution
  input_nchw  = np.transpose(input_nhwc, [0,3,1,2])
  output_nchw = simulated_nchw_conv_transpose(input_nchw, weights_nchw, strides)

Phase 4: Numerical Equivalence Check
  output_nhwc = np.transpose(output_nchw, (0,2,3,1))
  max_error   = max(|keras_output_nhwc - output_nhwc|)
  ASSERT max_error < 1e-5   # passes: 0.0 for all-ones input
```

The tolerance of `1e-5` was chosen deliberately. Machine epsilon for float32 is approximately 1.19 × 10■■. Due to summation tree reordering between NumPy (C backend) and C++ AVX/FMA instructions, `1e-5` accounts for accumulation drift while remaining tight enough to immediately flag structural stride or padding miscalculations. Relaxing to `1e-3` was explicitly rejected — it masks single-pixel padding misalignments.

5.3 C++ GTest Contract Test — TestRModelParserKeras.C

The C++ test exercises the full SOFIE pipeline from Keras model file to numerical output, verifying both AST topology and numeric correctness:

TestRModelParserKeras.C — CONV2D_TRANSPOSE GTest

```
TEST(RModelParser_Keras, CONV2D_TRANSPOSE) {
    constexpr float TOLERANCE = 1e-5f;
    std::vector<float> input(16, 1.0f);  // Batch=1, H=4, W=4, C=1

    TMVA::Experimental::RSofieReader r("KerasModelConv2DTranspose.keras");
    auto output = r.Compute(input);

    // True AST Topology Contract:
    auto operators = r.GetModel().GetOperators();
    EXPECT_EQ(operators.size(), 2);
    EXPECT_EQ(operators[0]->Type(), "Transpose");     // layout conversion
    EXPECT_EQ(operators[1]->Type(), "ConvTranspose"); // core operation

    // Numeric Equivalence (NCHW-aligned ground truth):
    PyRun_String("output = numpy.transpose(model(input_data).numpy(),"
                 "(0,3,1,2)).astype(numpy.float32)", ...);
    for (size_t i = 0; i < output.size(); ++i)
        EXPECT_LE(std::abs(output[i] - pOutput[i]), TOLERANCE);
}
```

§6 Lessons Learned

These are the things I now understand better having worked through this exercise — not just the mechanics, but the engineering philosophy behind them.

- **Memory contiguity is not optional at language boundaries:** The Python↔C++ bridge through pybind11 is a trust boundary. Python has garbage collection, reference counting, and strided memory views. C++ has raw pointers and static allocation. Passing a strided pointer across that boundary causes a segfault that is extremely difficult to diagnose. np.ascontiguousarray is the toll booth — it must be paid at every crossing.
- **Know where the arithmetic responsibility boundary lies:** The output_padding and padding computation bugs both came from the same mistake: trying to pre-compute in Python something the C++ ShapeInference engine was designed to own. The parser's job is to faithfully transport attributes, not to optimise them. Hardcoding static values for dynamic quantities produces bugs that only manifest on non-standard input sizes — the hardest kind to catch in testing.
- **True contract testing vs. black-box volume checks:** Checking that an output array has the right number of elements does not prove that the correct AST nodes were instantiated. In compiled ML inference pipelines, tests must explicitly query the operator stack (r.GetModel().GetOperators()) to verify the C++ engine actually built the requested computation graph. Black-box checks provide false confidence.
- **Never compare raw memory without layout alignment:** Comparing a flattened C++ NCHW output array directly against a flattened NumPy NHWC array produces large numerical errors that look like mathematical bugs but are actually layout mismatches. The dimension topologies must be aligned before any element-wise comparison. This seems obvious in retrospect but caused a complete uniform test failure.
- **Name sanitisation is a correctness concern, not just style:** The Clean_name() dot-erasure bug (Issue #21527) showed that seemingly cosmetic string processing decisions have real correctness consequences at the C++ variable declaration boundary. The fact that existing tests passed by coincidence — because those specific architectures happened to generate non-colliding name sequences

— is a reminder that coincidental test coverage is not the same as structural test coverage.

- **Infrastructure bugs are as important as logic bugs:** The CMake silent exclusion vulnerability (PR #21582) taught me that a test suite that is not running provides exactly zero correctness guarantees, and worse, provides false confidence that everything is green. Validating the test infrastructure itself — not just the code under test — is a necessary part of any serious software engineering effort.

§7 Relation to Existing SOFIE Infrastructure

This Python parser is designed to plug into the existing SOFIE Keras pipeline at the point where `RModelParser_Keras.cxx` dispatches to Python via `PyRunString` after identifying a Keras `Conv2DTranspose` layer type. The returned list is consumed by the C++ factory in the same way as dictionaries returned by existing Conv2D and Dense parsers — with the extension that the factory must iterate over a `PyList` rather than processing a single `PyDict`.

The `weight` and `bias` numpy arrays are passed to the corresponding `Initialize()` vectors via the pybind11 buffer protocol, which requires C-contiguous float32 layout. The `kernel_shape`, `strides`, `dilations`, `group`, and `autopad` keys map directly to constructor arguments of `ROperator_ConvTranspose`. The Transpose node uses the existing `ROperator_Transpose` with perm vector `[0, 3, 1, 2]`.

§8 Next Steps

Conv2DTranspose is one operator from Exercise 5. My plan for extending operator coverage follows the same pattern — identify layout requirements, map attributes, handle edge cases — but each new operator brings specific complexity:

- **BatchNorm2D (Keras):** Running statistics extraction, affine=False fallback synthesis, epsilon boundary ownership — analogous to the PyTorch BatchNorm2d work already completed.
- **DepthwiseConv2D:** Group convolution semantics require correct group count extraction and channel-splitting weight layout.
- **UpSampling2D:** Resize operator mapping; nearest-neighbour vs bilinear interpolation mode dispatch.
- **GlobalAveragePooling2D:** Spatial reduction operator; requires kernel_shape = [H, W] from input shape inference.

> All code is staged on my personal development branch and is ready for upstream review. I have not opened a PR to the ROOT project per the exercise instructions.

Aditya Rathore
GSoC 2026 Candidate — SOFIE PyTorch/Keras Parser
GitHub: @AdityaDRathore · adityarathore7067@gmail.com
Submitted March 2026